

Distinct Subsequences — Dynamic Programming

Complete step-by-step trace & matrix breakdown for $s = \text{"rabbbit"}$ and $t = \text{"rabbit"}$

1. Algorithm Logic & State Recurrence

Let $dp[i][j]$ be the number of distinct subsequences of suffix $s[i \dots n-1]$ matching suffix $t[j \dots m-1]$. Because matching starts from suffixes, the algorithm computes cells bottom-up, starting from the end of both strings down to $(0, 0)$.

Base Case (Column m):

$dp[i][m] = 1$ for all $0 \leq i \leq n$

An empty target suffix is matched exactly once by deleting all remaining characters of s.

Transitions (Two Options):

1. Skip $s[i]$: $dp[i][j] = dp[i+1][j]$

2. Match (if $s[i] == t[j]$): $+ dp[i+1][j+1]$

2. Full 2D Dynamic Programming Matrix

Evaluation proceeds from row 6 down to 0, and column 5 down to 0. Final result is at $dp[0][0] = 3$.

s \ t	r j=0	a j=1	b j=2	b j=3	i j=4	t j=5	"" j=6
i=0 'r'	3	0	0	0	0	0	1
i=1 'a'	0	3	0	0	0	0	1
i=2 'b' (1st)	0	0	3	1	0	0	1
i=3 'b' (2nd)	0	0	0	2	0	0	1
i=4 'b' (3rd)	0	0	0	1	0	0	1
i=5 'i'	0	0	0	0	1	0	1
i=6 't'	0	0	0	0	0	1	1
i=7 ""	0	0	0	0	0	0	1

Base Cases (1): Target string fully matched

Active Branch: Valid partial submatches

Final Answer (3): Complete matching subsequences

3. Trace: Why is the Answer 3? (The 3 'b's Dilemma)

String s contains three 'b's, but target t requires only two. We choose 2 out of 3, yielding $C(3, 2) = 3$ ways:

Cell	Characters	Calculation	Explanation
dp[4][3]	$s[4]='b' == t[3]='b'$	$dp[5][3] + dp[5][4] = 0 + 1 = 1$	3rd 'b' matches 2nd 'b' of t.
dp[3][3]	$s[3]='b' == t[3]='b'$	$dp[4][3] + dp[4][4] = 1 + 1 = 2$	Can either skip 2nd 'b' (1 way) or match it (1 way).
dp[2][2]	$s[2]='b' == t[2]='b'$	$dp[3][2] + dp[3][3] = 1 + 2 = 3$	Combines options for 1st 'b', propagating 3 valid combinations up to dp[0][0].

4. Space Optimization Note

Notice that row i only references row $i+1$. The 2D table of size $O(n \times m)$ can be optimized to a single 1D array of size $O(m)$ by iterating backwards across columns.